,

Gather GO

要件定義書

作成者	志水　長谷川　佐藤　新屋
作成日	2025/11/14
最終更新日	2026/1/15
文書番号	

麻生情報ビジネス専門学校

目次

1. システム概要 3
1-1. システム化の背景 3
1-2. システム化の目的 3
1-3. システム化の範囲 3
1-4. 現行のフロー 3
1-5. システム化後のフロー 3
2. 機能要件 3
2-1. 機能一覧 3
2-1-1. 機能 1 ..... エラー! ブックマークが定義されていません。
2-1-2. 機能 2 ..... エラー! ブックマークが定義されていません。
2-1-3. 機能 3 ..... エラー! ブックマークが定義されていません。
2-1-4. 機能 4 ..... エラー! ブックマークが定義されていません。
2-2. 画面レイアウト・画面イメージ 5
3. 非機能要件 5
3-3. 性能・拡張性 ..... エラー! ブックマークが定義されていません。
3-5. 可用性 ..... エラー! ブックマークが定義されていません。
3-6. 移行性 ..... エラー! ブックマークが定義されていません。
3-7. セキュリティ ..... エラー! ブックマークが定義されていません。
3-10. 運用・保守 ..... エラー! ブックマークが定義されていません。

1. システム概要

1-1. システム化の背景
大人数での飲み会・イベント開催時において、日程調整や参加確認、予約手配などの幹事業務が個人に集中している。
この負担を軽減し、グループチャット内で自動的に調整・集計を行う仕組みを作る。

1-2. システム化の目的
チャットボットを導入し、LINE グループ内で幹事業務を代行。

1-3. システム化の範囲
空き日確認、投票作成、店舗検索。

1-4. 現行のフロー
LINE グループ作成 → 候補日投票 → 集計→
候補店を提案 → 店の予約 → 予約完了を共有

1-5. システム化後のフロー
LINE グループ作成→カレンダー自動集計・最適日を表示 →
候補店を提案 → 店の予約→予約完了を共有

2. 機能要件

2-1. 機能一覧

番号	機能名	概要
1	Google 認証	Google Calendar と連携しユーザーの予定情報を取得できるようにする
2	空き日程取得	直近の日程で空いている日を自動抽出
3	日程投票	メンバーに日程の投票を依頼し、結果を集計
4	条件解析（AI）	Gemini による自然言語理解で検索条件を抽出
5	HotPepper 検索	キーワード・位置情報から店舗検索
6	店舗カルーセル表示	LINE リッチメッセージで店舗一覧を表示
7	投票機能	選択肢を元に投票を作成
8	投票実行	各ユーザー1 回の投票（重複防止）
9	投票集計	最多票の選択肢を決定
10	ON/OFF 管理	@on/@off による bot 有効/無効切替
11	履歴保存	会話履歴を保存し、AI 応答に利用

番号	機能名	詳細
2-1-1	Google 認証	・ Google OAuth による認証フローを実装 ・ユーザーの Google Calendar 読み取り権限を取得 ・認証情報は安全に保存し、必要に応じて更新 ・認証エラー時の再認証フローを提供
2-1-2	空き日程取得	・ Google Calendar API から予定一覧を取得。 ・本日～一定期間（例：7 日 or 14 日）の空き枠を自動解析。 ・ LINE 上で空き日程リストを提示。 ・解析ロジックは 30 分 or 1 時間単位など選択可能。
2-1-3	日程投票	・候補日を LINE メッセージで投票形式で提示。 ・ユーザーごとに 1 回のみ回答可能（重複防止）。 ・投票状況を DynamoDB に保存。 ・投票締切後、自動で最多得票を決定。 ・締切前でも途中経過を参照可能。
2-1-4	条件解析（AI）	・「博多で 3000 円以内の居酒屋」など自然文を解析。 ・抽出要素：エリア、予算、ジャンル、人数、時間帯など。 ・曖昧な記述にも対応（例：「ちょっと安め」「焼き鳥系」など）。 ・機械学習モデルによる継続学習を想定。
2-1-5	HotPepper 検索	・ HotPepper API を利用し店舗データを取得。 ・条件（予算・駅名・キーワード・カテゴリ）に基づく検索。 ・ API エラー時のリトライ・例外処理。 ・最大取得件数・並び順（おすすめ順/距離順）の設定。
2-1-6	店舗カルーセル表示	・ Flex Message によるカード形式の一覧表示。 ・店舗名、写真、予算、アクセス、URL、MAP を表示。 ・次ページ送り対応。 ・ユーザーのタップで詳細画面 or 外部リンクへ遷移。

2-1-7	投票作成	・タイトルと選択肢（2〜任意数）を登録。 ・ DynamoDB に保存。 ・ LINE Bot から `/vote create` などで開始できる。 ・入力不足チェック（タイトル未入力、選択肢不足）。
2-1-8	投票実行	・ユーザーは選択肢をタップして即時投票。 ・同一ユーザーの重複投票は自動拒否。 ・投票記録を DynamoDB に保存。 ・投票後に確認メッセージを返す。
2-1-9	投票集計	・ DynamoDB から全投票結果を集計。 ・票数の多い選択肢を自動判定。 ・途中経過、最終結果の表示に対応。 ・同数時の扱い（複数同率）も定義。
2-1-10	ON/OFF 管理	・@on → AI 応答・機能有効 ・@off → AI 応答停止（キーワードコマンドのみ受付） ・状態を DynamoDB に保存。 ・グループ・個別で別々の状態を管理。
2-1-11	履歴保存	・直近の会話ログを DynamoDB に保存。 ・ AI へのプロンプトに再利用。 ・履歴数の上限（例：20 件）。 ・プライバシー保護のため削除コマンドにも対応。

2-2. 画面レイアウト・画面イメージ

3. 非機能要件

性能・拡張性

同時アクセス数：最大 100 グループまで対応 応答時間：2 秒以内を目標 将来的にグループ拡張可能な設計を検討

可用性

24 時間稼働を基本とし、障害時は自動再起動対応 クラウド（AWS Lambda）上で運用、冗長化を実施

移行性

新規導入のため移行データなし テストグループで試験導入 → 本運用へ段階移行

セキュリティ

LINE 認証を利用し、不正アクセスを防止 API キーは環境変数管理 インシデント発生時は管理者メールへ自動通知

運用・保守

週 1 回のログ確認 データバックアップを自動実施（1 日 1 回） 定期的に API 更新対応を行う